Assignment 2 – Formal Specification: Report draft (Part A)

Draft of the control-process parts of the report (2IX20). Rewrite in your own words and make sure you can explain every label. Numbers were measured with TLC 2.19 (= Toolbox 1.7.4) in a sandbox: re-measure on your own machine before submitting. Parts for person B (truck write-up, property table, declined cards, maximal compressed trash) are not included.

1. Control process, single trash bin

1.1 High-level structure

The control process is one infinite loop (`ControlLoop`). In every iteration it handles exactly one event, selected with `either … or`. The three events have a fixed priority, which is enforced through the guards of the branches:

1. **Truck arrived**: the control sends `start_emptying` to the truck.
2. **Bin full and truck idle**: the control sends `request` to the truck.
3. **Scan available**: the control handles the next scan, but only if neither event 1 nor event 2 is enabled.

The priorities are needed because `fair process` only guarantees that the control process takes steps, not that every branch of the `either` is eventually taken. Without priorities the control could answer an endless stream of declined scans and never serve the truck. `TrashEmptied` and `TruckEmpties` would then fail even under weak fairness.

The control communicates with the other processes as follows: scans arrive in `scans`; the card is checked through `serverRequests` and `serverResponses`; the answer goes to the user via `permissions`; the bin is operated through `binCommand` (and observed through `binSensor`); the truck is handled through `truckCommand`.

1.2 Handling a scan

The control reads the head of `scans`, asks the server whether the card is valid, and then grants permission only if the card is valid, no truck is reserved for the bin, and the remaining capacity is at least `MaxUserTrash`. The capacity condition follows Note 10: the control cannot know how much the user carries, but it knows the user carries at most `MaxUserTrash`.

If permission is granted, the deposit is processed in a fixed order:

Label	Action	Reason
`ControlUnlock`	unlock outer door	the user can only open an unlocked door; unlocking only on grant means an unauthorised user never meets an unlocked door
`ControlGrant`	write `granted = TRUE`	only after the unlock has finished

Label	Action	Reason
`ControlAwaitClosed`	wait for the door sensor, then lock the outer door	the door must be locked before the trap door moves
`ControlOpenTrap`	open trap door	the trash falls into the main compartment
`ControlExtendRam`	extend ram	the trap door is open, so the mechanism is not destroyed; the trash is compressed
`ControlRetractRam`	retract ram	the trap door may only be changed while the ram is retracted
`ControlCloseTrap`	close trap door	emptying requires a closed trap door

If permission is not granted (invalid card, truck on its way, or not enough room), the control writes `granted = FALSE` immediately, and the user simply scans again.

```
fair process controlProcess = Control
variables
  scan = [user |-> 0, bin |-> 0],
  resp = [user |-> 0, permission |-> FALSE];
begin
  ControlLoop:
    while TRUE do
      either
        \* (1) Truck arrived: bin is safe (control is between two deposits)
        await truckCommand.command = "arrived" /\ binCommand.command = "finished";
        truckCommand := [command |-> "start_emptying", bin |-> 1];
      or
        \* (2) Bin is full and the truck is idle: request it
        await TruckIdle /\ BinFull;
        truckCommand := [command |-> "request", bin |-> 1];
      or
        \* (3) Handle the next scan
        await scans /= <<>> /\ truckCommand.command /= "arrived"
              /\ ~(TruckIdle /\ BinFull);
        read(scans, scan);
        write(serverRequests, [user |-> scan.user]);
  ControlAwaitServer:
        read(serverResponses, resp);
        assert resp.user = scan.user;
        if resp.permission /\ TruckIdle /\ TrashFitsCapacity(MaxUserTrash) then
  ControlUnlock:
          await binCommand.command = "finished";
          binCommand := [command |-> "change_outer_lock", open |-> TRUE];
  ControlGrant:
          await binCommand.command = "finished";
          write(permissions, [user |-> scan.user, bin |-> scan.bin, granted |->
TRUE]);
  ControlAwaitClosed:
          await binSensor.sensor = "outer_door_closed" /\ binCommand.command =
"finished";
```

```
          binSensor := [sensor |-> "idle"];
          binCommand := [command |-> "change_outer_lock", open |-> FALSE];
  ControlOpenTrap:
          await binCommand.command = "finished";
          binCommand := [command |-> "change_trap_door", open |-> TRUE];
  ControlExtendRam:
          await binCommand.command = "finished";
          binCommand := [command |-> "change_ram", open |-> TRUE];
  ControlRetractRam:
          await binCommand.command = "finished";
          binCommand := [command |-> "change_ram", open |-> FALSE];
  ControlCloseTrap:
          await binCommand.command = "finished";
          binCommand := [command |-> "change_trap_door", open |-> FALSE];
        else
          write(permissions, [user |-> scan.user, bin |-> scan.bin, granted |->
FALSE]);
        end if;
      end either;
    end while;
end process;
```

1.3 Modelling decisions

- **Every command waits for the previous one.** Each command to the bin is preceded by `await binCommand.command = "finished"` in the same atomic step. Otherwise the bin process, which later sets `binCommand.command := "finished"` (label `BinCommandFinished`), could overwrite a newer command.
- **When to request a truck.** The truck is requested when the bin cannot accept another maximal deposit, i.e. when `BinFull == ~TrashFitsCapacity(MaxUserTrash)`. From that moment the control would have to decline every scan anyway. The truck is therefore requested when needed and not after every deposit.
- **Scans during truck waiting.** The control never blocks on the truck. While a truck is on its way, scans are still read and answered (declined).
- **Truck handshake.** `emptied` (idle) → `request` → `arrived` → `start_emptying` → `emptied`. The control answers `arrived` only between two deposits, so the bin is always safe at that moment (door locked, trap door closed, ram retracted, nothing in the outer door). The truck then sends `empty` itself.
- **Door locked by default.** The outer door is unlocked only between the grant and the moment the user closes it again.
- **Few labels.** The lock command is merged into `ControlAwaitClosed` and the decline branch needs no label of its own. Fewer labels give fewer interleavings and a smaller state space.

2. Control process, multiple trash bins

2.1 Main adaptations

The structure of the control (loop, three prioritised events, same deposit sequence) is unchanged. The adaptations are:

- All bin variables are indexed by the bin of the scan (`scan.bin`), permissions are written to the queue of the scanning user (`permissions[scan.user]`), and each truck has its own slot `truckCommand[t]`.
- The status of bins and trucks is **derived** from `truckCommand` instead of being stored in extra variables:

```
BinFull(bin) == ~TrashFitsCapacity(bin, MaxUserTrash)
BinReserved(bin) == \E t \in Trucks :
      /\ truckCommand[t].bin = bin
      /\ truckCommand[t].command \in {"request", "arrived", "start_emptying"}
NeedyBins == {b \in Bins : BinFull(b) /\ ~BinReserved(b)}
ArrivedTrucks == {t \in Trucks : truckCommand[t].command = "arrived"}
ReadyTrucks == {t \in ArrivedTrucks :
      binCommand[truckCommand[t].bin].command = "finished"}
PickBin == CHOOSE b \in NeedyBins : TRUE
PickTruck == CHOOSE t \in AvailableTrucks : TRUE
PickReady == CHOOSE t \in ReadyTrucks : TRUE
```

The three events become:

```
either
  await ReadyTrucks /= {};
  truckCommand[PickReady].command := "start_emptying";
or
  await NeedyBins /= {} /\ AvailableTrucks /= {};
  truckCommand[PickTruck] := [command |-> "request", bin |-> PickBin];
or
  await scans /= <<>> /\ ArrivedTrucks = {}
        /\ ~(NeedyBins /= {} /\ AvailableTrucks /= {});
  read(scans, scan);
  ...
```

2.2 Modelling decisions

- **When to request a truck.** Same strategy as for one bin: a truck is requested for a bin when it is full and no truck is reserved for it yet. A bin that has a truck reserved declines all scans, other bins are unaffected.
- **Which truck to request.** The control picks the lowest-numbered needy bin and the lowest-numbered idle truck, deterministically (`CHOOSE`). A non-deterministic choice would only multiply the state space without adding behaviour that matters for the properties. Several trucks can be on their way at the same time.
- **Parallel handling.** Truck handling and scan handling are interleaved, so scans are answered while trucks drive. Deposits themselves are handled one at a time, and the scans of the other users wait in the FIFO queue. This keeps the argument for safety simple (when a truck has arrived, the control is between two deposits, so the bin is safe) and keeps the model within the 5-minute limit.
- **No extra control state.** Because status is derived from `truckCommand`, the control only has local variables for the scan it is currently handling.
- **Changes compared to the single model.** Only the indexing described above, plus the derived definitions. No new communication variables were needed.

3. Verification (control-related results)

All processes are declared as `fair process` (weak fairness). For all configurations TLC reports no deadlock, and the invariants `TypeOK`, `MessagesOK`, `CapacitiesRespected`, `TrapNotDestroyed` and `CapacityNotExceeded` hold.

Model	Configuration	Properties	Distinct states	Time
single	1 bin, 1 user, 1 truck, capacity 7, max 4	all hold	7,052	4 s
multiple	Bins: 2/1/1, capacity 3, max 2	all hold	75,424	37 s
multiple	Users: 1/2/1, capacity 3, max 2	`UserTrash` and `UserTrashDeposited` violated, others hold	104,384	47 s
multiple	Trucks: 2/1/2, capacity 2, max 1	all hold	118,102	80 s

Without fairness, the liveness properties fail with a stuttering counter-example (checked for the single model), which is why weak fairness of all processes is assumed.

Scalability. Both Bins and Trucks finish well within 5 minutes. The control was kept small on purpose: few labels, a deterministic choice of bin and truck, and no extra state variables.

4. Investigation: multiple users

For configuration **Users** (process ids: bin 1, users 2 and 3, truck 4), both `UserTrash` and `UserTrashDeposited` are violated.

Counter-example. TLC finds a lasso of 176 states whose loop starts at state 88. In every round, user 3 deposits twice; the second deposit makes the bin full. The control requests the truck, and user 2's scan is handled while the bin is reserved, so it is declined. User 2 scans again. After the truck has emptied the bin, user 3's scan is again at the head of the queue, user 3 deposits, and the round repeats. User 2 keeps its trash forever, so `userTrash[2] > 0` holds in every state of the loop. This violates `UserTrashDeposited` (it never deposits) and `UserTrash` (it never has no trash).

Fairness. The violation also occurs under weak fairness of all processes (the model's `Spec`) and under strong fairness of all processes (checked with a variant of `Spec` using `SF`). Fairness only guarantees that each process eventually takes a step. It does not control the order in which scans enter the queue relative to the truck cycle, and the scan step of user 2 is continuously enabled, so strong fairness adds nothing.

More trucks or bins. With a second truck (1 bin, 2 users, 2 trucks) `UserTrashDeposited` is still violated, so the problem is not a lack of emptying capacity. The run with 2 bins, 2 users and 1 truck did not finish within 280 s without finding a violation, so no claim is made for extra bins from the model checker.

Cause in the control process. A declined user has no memory: it has to scan again and is appended to the back of the FIFO queue, so nothing guarantees that it is served first once the bin is free. A fix would be to give declined users priority or to reserve the emptied bin for the first declined user.

With one user (Bins and Trucks) both properties hold, because there is no competitor.

5. Investigation: bin sensor

Experiment (temporary change, single model). In `ControlAwaitClosed` the condition `binSensor.sensor = "outer_door_closed" /\ binCommand.command = "finished"` was replaced by only `binCommand.command = "finished"`.

Result. TLC reports `Failure of assertion at line 171`, the assertion `~outerDoorLocked` of the bin for `change_outer_door`. In the trace, `binCommand` is already `finished` after the unlock, so right after `ControlGrant` the control continues and locks the door again before the user has even sent `change_outer_door`. When the user then tries to open the door, the bin finds it locked.

Explanation. `finished` only says that the last command to the bin has been executed. It does not say that the user has deposited and closed the door, because the user operates the outer door directly without the control. The sensor is the only signal for "the user has closed the door". The required behaviour (move the trash only after the user has closed the door) is therefore **not** preserved.

The model awaits the sensor together with `finished`: the bin sets the sensor in one step and `finished` in the next, so without the second condition the control could overwrite the still unfinished user command.

6. AI usage (part A, to be completed with person B)

Declare per section, as §6.2 requires. Suggested text: for Sections 1 and 2, a generative AI assistant was used to draft the control process (single and multiple), the truck process used for testing and draft property formulas; we reviewed the code, ran it in TLC and kept / changed the following: … For Sections 3 to 5, the assistant ran the model checks and drafted the explanations of the counter-examples, which we checked against the TLC traces. For the report text, the assistant was used to improve the language. Fill in what you actually kept, edited or rejected.